



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине «Разработка баз данных»

Практическое занятие № 8

Студенты группы *ИКБО-42-23 Голев С.С.*

(подпись)

Ассистент *Морозов Д.В.*

(подпись)

Отчет представлен «__» _____ 2025 г.

Москва 2025 г.

СОДЕРЖАНИЕ

ЗАДАНИЕ	3
ВЫПОЛНЕНИЕ ЗАДАНИЯ	5
Подготовка БД	5
Анализ и оптимизация	5
Демонстрация атомарной транзакции	7
Моделирование аномалии	8
Моделирование неповторяемого чтения	8
Устранение неповторяемого чтения	9

ЗАДАНИЕ

Для выполнения практической работы необходимо последовательно выполнить следующие шаги, адаптируя примеры из БД «Аптека» к вашей собственной базе данных.

Задание №1: хранение паролей (BYTEA)

1. Создать справочную таблицу ролей и таблицу системных пользователей (или модифицировать существующую).
2. В таблице пользователей предусмотреть поля для хранения «сырого» пароля (только для учебной демонстрации) и его хеша (тип BYTEA).
3. Сохранить хеши паролей пользователей, используя функцию хеширования digest.
4. Выполнить запрос, проверяющий соответствие введенного пароля сохраненному хешу. Одна проверка должна быть успешной, другая - нет (допустимо сделать это в одном запросе).

Задание №2: секционирование таблицы логов

1. Создать новую секционированную таблицу для хранения логов событий.
2. Использовать стратегию секционирования по диапазонам (RANGE) по полю даты.
3. Создать две секции для периодов:
 - 2-е полугодие 2024 (с 2024-07-01 по 2025-01-01).
 - 1-е полугодие 2025 (с 2025-01-01 по 2025-07-01).
4. Создать секцию по умолчанию, куда будут попадать все данные, выходящие за рамки указанных диапазонов (будущее время).

Задание №3: генерация данных

Написать скрипт на PL/pgSQL для генерации 20 000 записей в диапазоне дат, покрывающем созданные секции. Сгенерированные данные должны содержать JSON-структуру с различным набором полей для разных типов событий (например, чтобы логи «продажи» отличались по структуре от логов «входа»).

Вывести сгенерированные данные – показать запросом SELECT, что данные успешно созданы и физически распределились по разным таблицам секциям.

Задание №4: анализ и поиск по JSONB

Написать три запроса:

1. Фильтрация по значению – найти записи, где числовое поле внутри JSON удовлетворяет математическому условию (например, > 800), используя операторы извлечения $\rightarrow$ и $\rightarrow>$.

2. Поиск по вхождению – найти записи, содержащие точный фрагмент JSON-структуры (например, $\{"quantity": 5\}$), используя оператор $@>$.

3. Агрегация – посчитать сумму или среднее значение числового поля из JSON-документа.

Задание №5: модификация данных JSONB

Продемонстрировать изменение данных внутри JSON-поля:

1. Массовое добавление нового поля во все записи определенного типа.
2. Точечное изменение значения по ключу в конкретной записи.

ВЫПОЛНЕНИЕ ЗАДАНИЯ

Хранение паролей

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

ALTER TABLE access_data
ADD COLUMN IF NOT EXISTS password_raw TEXT,
ADD COLUMN IF NOT EXISTS password_hash BYTEA;

UPDATE access_data
SET
password_raw = password,
password_hash = digest(password, 'sha256')
WHERE password_hash IS NULL;

SELECT
id_access_data,
login,
password_raw,
encode(password_hash, 'hex') AS password_hash_hex,
(digest('9001001', 'sha256') = password_hash) AS ok_correct_password,
(digest('wrongpass', 'sha256') = password_hash) AS ok_wrong_password
FROM access_data
WHERE login = '2001001';
```

The screenshot shows a SQL IDE window titled 'access_data 1'. Below the SQL editor, a table view displays the results of the query. The table has 7 columns: id_access_data, login, password_raw, password_hash_hex, ok_correct_password, and ok_wrong_password. The first row shows the results for login '2001001'.

	id_access_data	login	password_raw	password_hash_hex	ok_correct_password	ok_wrong_password
1	1 001	2001001	9001001	2286cc874f67b04094f7c1a6263ef46b7b84e43923df50a967ffcd3e24085a6	[v]	[]

Рисунок 1 – Реализация хранения паролей

Секционирование таблицы логов

The screenshot displays a database IDE with the following SQL queries:

```
CREATE TABLE user_logs (  
    log_id BIGSERIAL,  
    created_at TIMESTAMPTZ NOT NULL,  
    actor_type TEXT NOT NULL CHECK (actor_type IN ('client','employee')),  
    actor_id INT NOT NULL,  
    event_data JSONB NOT NULL,  
    PRIMARY KEY (log_id, created_at)  
) PARTITION BY RANGE (created_at);  
  
CREATE TABLE user_logs_2024_h2 PARTITION OF user_logs  
FOR VALUES FROM ('2024-07-01') TO ('2025-01-01');  
  
CREATE TABLE user_logs_2025_h1 PARTITION OF user_logs  
FOR VALUES FROM ('2025-01-01') TO ('2025-07-01');  
  
CREATE TABLE user_logs_default PARTITION OF user_logs DEFAULT;  
  
CREATE INDEX IF NOT EXISTS idx_user_logs_created_at ON user_logs(created_at);
```

Below the queries, a statistics window titled "Статистика 1" is open, showing the following data:

Name	Value
Updated Rows	0
Execute time	0.119s
Start time	Mon Dec 22 00:54:16 MSK 2025
Finish time	Mon Dec 22 00:54:17 MSK 2025
Query	CREATE INDEX IF NOT EXISTS idx_user_logs_created_at ON user_logs(created_at)

Рисунок 2 – Реализация родительской секционированной таблицы и секций

Генерация данных

```
DO $$
DECLARE
    i INT;
    random_date TIMESTAMPTZ;
    ev TEXT;

    v_client_id INT;
    v_employee_id INT;

    v_order_id INT;
    v_rtl_id INT;

    v_ip TEXT;
    v_qty INT;
    v_total NUMERIC(10,2);

    payload JSONB;
BEGIN
    FOR i IN 1..20000 LOOP
        random_date :=
            '2024-07-01'::timestamptz
            + random() * ('2025-07-01'::timestamptz - '2024-07-01'::timestamptz);

        v_client_id := floor(random() * 6 + 201)::int;
        v_employee_id := floor(random() * 6 + 501)::int;

        v_order_id := floor(random() * 6 + 11001)::int;
        v_rtl_id := floor(random() * 6 + 801)::int;

        v_qty := floor(random() * 10 + 1)::int;
        v_total := round((random() * 1500)::numeric, 2);

        IF (i % 3 = 0) THEN
            ev := 'login';
            v_ip := '10.0.' || floor(random()*255)::int || '.' || floor(random()*255)::int;

            payload := jsonb_build_object(
                'event', ev,
                'ip', v_ip,
                'success', (random() > 0.1)
            );
        
```

Рисунок 3 – Реализация генерации данных, часть 1

```

INSERT INTO user_logs(created_at, actor_type, actor_id, event_data)
VALUES (random_date, 'client', v_client_id, payload);

ELSIF (i % 3 = 1) THEN
  ev := 'order';
  payload := jsonb_build_object(
    'event', ev,
    'details', jsonb_build_object(
      'ordering_id', v_order_id,
      'total', (SELECT total FROM ordering WHERE id_ordering = v_order_id),
      'delivery_method', (SELECT delivery_method FROM ordering WHERE id_ordering = v_order_id),
      'status', (SELECT status FROM ordering WHERE id_ordering = v_order_id)
    )
  );

  INSERT INTO user_logs(created_at, actor_type, actor_id, event_data)
  VALUES (random_date, 'client', v_client_id, payload);

ELSE
  ev := 'sale';
  payload := jsonb_build_object(
    'event', ev,
    'details', jsonb_build_object(
      'employee_id', v_employee_id,
      'ordering_id', v_order_id,
      'rtl_id', v_rtl_id,
      'quantity', v_qty,
      'total_sum', v_total
    )
  );

  INSERT INTO user_logs(created_at, actor_type, actor_id, event_data)
  VALUES (random_date, 'employee', v_employee_id, payload);
END IF;
END LOOP;
END $$;

```

Рисунок 4 – Реализация генерации данных, часть 2

● `SELECT '2024_h2' AS partition_name, count(*) FROM user_logs 2024 h2`
`UNION ALL`
`SELECT '2025_h1', count(*) FROM user_logs 2025 h1`
`UNION ALL`
`SELECT 'default', count(*) FROM user_logs default;`

<

пытат 1 ×

CT '2024_h2' AS partition_name, count(*) FROM user_logs_2024 | Введите SQL выражение что

A-Z partition_name	123 count	
2024_h2	10 106	
2025_h1	9 894	
default	0	

Рисунок 5 – Проверка

Анализ и поиск по JSONB

The screenshot shows a database interface with three SQL queries in the editor and a table of results below.

Query 1:

```
SELECT log_id, created_at, event_data
FROM user_logs
WHERE event_data ->> 'event' = 'sale'
AND (event_data -> 'details' ->> 'total_sum')::numeric > 800
LIMIT 5;
```

Query 2:

```
SELECT log_id, created_at, event_data
FROM user_logs
WHERE event_data @> '{"details": {"quantity": 5}}'
LIMIT 5;
```

Query 3:

```
SELECT
AVG((event_data -> 'details' ->> 'total_sum')::numeric)::numeric(10,2) AS avg_sale_sum
FROM user_logs
WHERE event_data ->> 'event' = 'sale';
```

Table Results:

	log_id	created_at	event_data
1	8	2024-07-26 08:00:59.994 +0300	{"event": "sale", "details": {"rtl_id": 805, "quantity": 7, "total_sum": 1272.70, "employee_id": 504, "ordering_id": 11004}}
2	11	2024-12-02 06:29:16.702 +0300	{"event": "sale", "details": {"rtl_id": 802, "quantity": 4, "total_sum": 955.28, "employee_id": 504, "ordering_id": 11004}}
3	29	2024-08-05 20:37:20.622 +0300	{"event": "sale", "details": {"rtl_id": 804, "quantity": 3, "total_sum": 1282.43, "employee_id": 501, "ordering_id": 11006}}
4	32	2024-07-23 23:54:33.569 +0300	{"event": "sale", "details": {"rtl_id": 802, "quantity": 8, "total_sum": 884.10, "employee_id": 505, "ordering_id": 11002}}
5	56	2024-08-11 04:10:27.273 +0300	{"event": "sale", "details": {"rtl_id": 805, "quantity": 9, "total_sum": 811.27, "employee_id": 505, "ordering_id": 11005}}

Рисунок 6 – Фильтрация по числу

```

SELECT log_id, created_at, event_data
FROM user_logs
WHERE event_data ->> 'event' = 'sale'
AND (event_data -> 'details' ->> 'total_sum')::numeric > 800
LIMIT 5;

SELECT log_id, created_at, event_data
FROM user_logs
WHERE event_data @> '{"details": {"quantity": 5}}'
LIMIT 5;

SELECT
    AVG((event_data -> 'details' ->> 'total_sum')::numeric)::numeric(10,2) AS avg_sale_sum
FROM user_logs
WHERE event_data ->> 'event' = 'sale';

```

log_id	created_at	event_data
2	2024-08-03 05:32:16.538 +0300	{"event": "sale", "details": {"rtl_id": 805, "quantity": 5, "total_sum": 222.66, "employee_id": 505, "ordering_id": 11002}}
227	2024-10-28 01:41:59.205 +0300	{"event": "sale", "details": {"rtl_id": 803, "quantity": 5, "total_sum": 421.58, "employee_id": 506, "ordering_id": 11003}}
332	2024-10-11 23:53:32.224 +0300	{"event": "sale", "details": {"rtl_id": 802, "quantity": 5, "total_sum": 262.72, "employee_id": 506, "ordering_id": 11003}}
368	2024-12-07 16:05:35.852 +0300	{"event": "sale", "details": {"rtl_id": 806, "quantity": 5, "total_sum": 1460.72, "employee_id": 501, "ordering_id": 11001}}
494	2024-11-16 00:26:24.625 +0300	{"event": "sale", "details": {"rtl_id": 801, "quantity": 5, "total_sum": 743.70, "employee_id": 502, "ordering_id": 11006}}

Рисунок 7 – Поиск по вхождению

```

SELECT log_id, created_at, event_data
FROM user_logs
WHERE event_data ->> 'event' = 'sale'
AND (event_data -> 'details' ->> 'total_sum')::numeric > 800
LIMIT 5;

SELECT log_id, created_at, event_data
FROM user_logs
WHERE event_data @> '{"details": {"quantity": 5}}'
LIMIT 5;

SELECT
    AVG((event_data -> 'details' ->> 'total_sum')::numeric)::numeric(10,2) AS avg_sale_sum
FROM user_logs
WHERE event_data ->> 'event' = 'sale';

```

avg_sale_sum
753,14

Рисунок 8– Рассчёт средней суммы

Модификация данных JSONB

```
● UPDATE user_logs
  SET event_data = event_data || '{"source":"web"}'::jsonb
  WHERE event_data ->> 'event' = 'sale';

● UPDATE user_logs
  SET event_data = jsonb_set(event_data, '{success}', 'false'::jsonb)
  WHERE log_id = (
    SELECT min(log_id)
    FROM user_logs
    WHERE event_data ->> 'event' = 'login'
  );

● SELECT log_id, event_data
  FROM user_logs
  WHERE event_data ->> 'event' = 'login'
     AND event_data ->> 'success' = 'false'
  LIMIT 1;
```

r_logs 1 ×

SELECT log_id, event_data FROM user_logs WHERE event_data ->> Введите SQL выражение чтобы с

log_id	event_data
21	{"ip": "10.0.251.223", "event": "login", "success": false}

Рисунок 9– Модификация данных